

rtk for codescout: evaluation report

Should we adopt rtk-ai/rtk (Rust Token Killer) in front of codescout's `run_command`? What we measured, how, and what we recommend.

Date: 2026-09-24 · rtk version tested: v0.49.0 (released 2026-09-11) · codescout: branch `experiments @ 36999188`

1. Summary

Recommendation: do not adopt rtk as a wrapper around codescout's `run_command`.

The saving it would add is small: an upper bound of **0.63% of all codescout tool output** (about 1.2K tokens per session). That is because codescout already buffers every output larger than ~10 KB behind a short summary. rtk also **disables two codescout safety guards** (the empty-test-selection warning and the IL-3 pipe gate) and **removes content by default** (commit bodies, diff hunks, grep indentation).

Worth porting instead: summarising small, inline cargo test output, which codescout currently returns raw. rtk saves 88% on those calls. Implemented inside codescout, it keeps the raw output retrievable and keeps the guards working.

Question	Answer (measured)
Does rtk's Claude Code hook work with codescout?	No. It rewrites only native Bash tool calls. Native Bash is denied on all our profiles (since 2026-09-20), and the hook never touches MCP tools such as <code>run_command</code> .
How much of our shell traffic would rtk rewrite?	60.9% of <code>run_command</code> calls, 63.3% of bytes returned.
How much would it save?	At most 0.97 MB/month, which is 0.63% of the 154 MB of codescout tool output.
Does it break anything?	Yes. It silences the empty-test-selection warning, bypasses the IL-3 pipe gate (a failing suite reported exit 0), and removes content from <code>git log</code> / <code>git diff</code> / <code>grep</code> / <code>clippy</code> output.

2. What rtk is and how it would plug in

rtk is a single Rust binary (Apache-2.0, ~82k GitHub stars) that sits between an AI agent and the shell and filters command output before the model reads it. Its README claims 60-90% fewer tokens on common dev commands. It does this by filtering noise, grouping, truncating, and collapsing repeated lines. `rtk init -g` installs a Claude Code PreToolUse hook that rewrites Bash commands, e.g. `git status` becomes `rtk git status`.

Three features made offline measurement possible:

- `rtk rewrite "<cmd>"` prints the rtk form of a command, or exits 1 if rtk has no equivalent. It is the same function the hook uses.
- `rtk pipe -f <filter>` applies a filter to stdin, so previously recorded outputs can be re-filtered without re-running anything.
- `rtk discover` scans Claude Code transcripts for Bash commands rtk could have compressed.

Integration path for codescout. The stock hook cannot reach us, so adopting rtk would mean a custom hook that calls `rtk rewrite` on the command argument of `mcp__codescout__run_command`, or codescout prefixing commands itself. Either way, codescout would then receive rtk's filtered text instead of the real output. That matters for the findings in section 5.

3. How we tested

We used four complementary methods. The first two use **real recorded traffic** rather than a synthetic benchmark. The last two check **fidelity**, which byte counts cannot show.

#	Method	What it measures	Input
1	Corpus coverage	Which recorded commands rtk would rewrite at all	All run_command calls in codescout's usage . db, 2026-08-25 to 2026-09-24: 23,641 calls, 30.5 MB returned, 206 sessions
2	Offline replay	Bytes before/after rtk on outputs the model actually saw	Recorded outputs codescout returned inline (not buffered), single-segment commands only, piped through rtk pipe -f with the matching filter
3	Live A/B pairs	Raw vs rtk <cmd> on today's tree, plus content inspection	14 commands (git, grep, ls, wc) on the codescout checkout
4	Guard probes	Whether rtk output still triggers codescout's own safety checks	A throwaway cargo crate (45 tests: pass, fail, should_panic, ignored; one clippy warning), run raw and through rtk, both via codescout's run_command

Why bytes, and why only inline outputs. codescout returns an output in full when it is under ~10 KB. Above that it stores the full output in an @cmd_* buffer and returns a summary of at most ~3 KB. For buffered calls the model never saw raw output, so rtk's reduction of raw output is not the right baseline. For inline calls, the recorded output is exactly what the model read. Tokens are estimated as bytes / 4, the same convention rtk and codescout both use.

Hygiene: the rtk binary was the official release tarball, checked against the published sha256 (7278231d . . . 0c8f). Telemetry was disabled (RTK_TELEMETRY_DISABLED=1). Everything ran from a scratch directory, and no hook was installed.

4. Results

4.1 Coverage: what rtk would touch

Of 23,641 recorded run_command calls, rtk rewrite would rewrite **14,401 (60.9%)**, accounting for **19.3 MB (63.3%)** of the bytes codescout returned. The largest rewritable families:

rtk subcommand	calls	bytes returned	of which buffered by codescout
cargo test	2,936	6,097,483	761
grep -n	1,129	1,659,020	38
git diff	797	1,326,628	58
git log	789	981,602	7
git status	1,074	790,411	9
git show	244	640,443	27
git add	882	616,387	1
grep -E	514	608,286	9
ls -la	286	431,036	7
cargo clippy	449	372,955	98

Not rewritable (39.1% of calls): echo, python3, sed, shell loops, project scripts such as ./scripts/gate.sh, and queries against codescout's own @cmd_* buffers.

4.2 Offline replay: savings on outputs the model actually read

rtk filter	calls	bytes before	bytes after	saved	reliability
cargo-test	319	749,254	86,579	88.4%	Reliable: live rtk behaves the same (2,224 to 437 B on the fixture)
git-diff	333	671,474	473,591	29.5%	Upper bound (see below)
grep	287	297,591	278,934	6.3%	Reliable
git-log	111	106,693	28,923	72.9%	Upper bound: live rtk passes explicit --format through (-1%)
git-status	220	87,040	86,812	0.3%	Reliable: agents already use --short
find	24	24,007	13,506	43.7%	Reliable
Total	1,294	1,936,059	968,345	50.0%	~242K tokens/month, upper bound

Why two rows are upper bounds. Pipe mode applies rtk's default-format filter no matter which flags the original command used. We re-ran two outliers live. Recorded `git log -2 --format='===%n%B'` fell from 3,818 to 274 B in pipe mode, but live `rtk git log -2 --format=%B` went from 2,882 to 2,854 B: it respects the requested format. Likewise `git show --stat --format=...` went to 0 B in pipe mode (the git-diff filter misread it) but passed through unchanged live.

4.3 What the saving amounts to

Quantity	Value
All codescout MCP tool output, same 30 days	154.1 MB (~38.5M tokens)
of which <code>run_command</code>	30.6 MB (20%)
Maximum rtk saving (inline outputs, section 4.2)	0.97 MB = 0.63% of all tool output
Reliable part (cargo-test only)	0.66 MB = 0.43%
Per session (206 sessions)	~4.7 KB, about 1.2K tokens
rtk's own estimate from Bash-era transcripts (<code>rtk discover</code> , 248 sessions)	~1.6M tokens "saveable" across 19,445 Bash commands. This path no longer exists now that Bash is denied.

4.4 Live A/B pairs on the current tree

Command	raw B	rtk B	saved	What rtk removed
<code>git status</code>	997	514	48%	Git's hint text; rewritten as porcelain-like lines. No information lost.
<code>git log -20</code>	44,861	7,376	83%	Commit bodies cut to 3 lines + "[+N lines omitted]"; subjects cut at ~110 chars
<code>git log --oneline -20</code>	1,845	1,808	2%	Long subjects truncated
<code>git log -5 --stat</code>	10,080	10,080	0%	Nothing (passthrough)
<code>git diff (dirty tree)</code>	299,041	61,273	79%	Per-hunk caps ("702 additions truncated"); last file's hunks dropped entirely
<code>git diff --stat</code>	625	624	0%	Nothing
<code>git show HEAD</code>	5,862	4,565	22%	Diff compaction
<code>git show HEAD~3</code>	11,842	10,359	12%	Diff compaction
<code>grep -rn RecoverableError src/tools</code>	43,431	17,635	59%	Indentation stripped, long lines truncated, 22 files hidden behind rtk recall
<code>grep -rn "fn call_content" src</code>	1,060	1,060	0%	Nothing
<code>ls -la src/tools</code>	2,050	727	64%	Permissions/owner columns
<code>ls docs/issues</code>	5,393	3,916	27%	Layout
<code>wc -l (2 files)</code>	85	58	31%	Padding
<code>cargo test (fixture, 1 failure)</code>	2,224	437	80%	Passing test lines and compiler warnings; failure panic kept
<code>cargo clippy (fixture)</code>	944	179	81%	Lint names and the help: fix suggestions

A 15th pair (`find`) was excluded because the glob was unquoted and returned nothing in both arms.

4.5 Interaction with codescout's buffering

codescout already caps what the model sees for large outputs, so rtk's large-output savings mostly don't reach the model. Sometimes they make things worse:

Command	Under codescout today	Under codescout + rtk
<code>git log -20</code> (44.9 KB raw)	Buffered; the model sees a head/tail summary of at most ~3 KB, and the full log stays in <code>@cmd_*</code> (lossless)	rtk output is 7.4 KB, under the 10 KB threshold, so the model sees all 7.4 KB , and it is missing the commit bodies
<code>git diff</code> (299 KB raw)	Buffered; ~3 KB summary; full diff queryable	Still buffered (61 KB); same ~3 KB summary, but the buffer now holds the truncated diff
<code>cargo test --workspace</code> (~830 KB raw)	Buffered; the summary is about 120 B on a green run (passed/ignored counts)	Similar size; no gain

5. Fidelity and safety findings

Each finding below was observed live via codescout's `run_command`, not inferred from the source.

Finding	Evidence	Impact
Empty-test-selection warning silenced	Raw <code>cargo test no_such_test</code> prints <code>test result: ok. 0 passed; ... 45 filtered out</code> and codescout attaches its <code>empty_test_selection</code> warning. Through rtk the output is <code>cargo test: 0 passed, 45 filtered out</code> and no warning is attached .	A test filter that matches nothing reads as a pass. This is the incident the guard was built for.
IL-3 pipe gate bypassed; exit code masked	<code>cargo test tail</code> is blocked by IL-3. <code>rtk cargo test 2>&1 tail -2</code> is not blocked (the gate reads the left-hand side as rtk), and <code>run_command</code> reported <code>exit_code: 0</code> for a suite with a failing test.	Reopens the masked-failure hazard IL-3 exists to prevent.
Lossless buffer contract broken	rtk stores removed content in its own store (<code>rtk recall <hash></code>). codescout's <code>@cmd_*</code> buffer would hold the filtered text.	"Query the buffer instead of re-running" stops being safe; agents would have to learn a second recall mechanism.
Commit bodies cut	<code>git log</code> keeps 3 body lines per commit.	In this repo, commit bodies carry measurements, rationale and Session-Id trailers.
Diffs cut	Per-hunk caps, and trailing files are dropped with "(more changes truncated)".	Reviews and patch-id work on incomplete diffs.
grep lines altered	Leading whitespace stripped; long lines end in "...".	A matched line can no longer be pasted into an edit's <code>old_string</code> .
clippy detail dropped	Lint names and <code>help</code> : suggestions removed; only grouped messages and locations remain.	The model loses the suggested fix.
<i>Works correctly</i>	Exit codes preserved (101 on failure); failure panics kept, with left/right values.	-

6. Caveats and limits of this evaluation

- One machine, one month, one project (codescout itself). A team with a different command mix, such as heavy JS/pytest use, could see different coverage.
- The offline replay covers inline, single-segment commands only (1,294 calls). Chained or piped commands were excluded because their recorded output is not what rtk would see.
- Pipe mode overstates git-log/git-diff savings (section 4.2), so the 0.63% total is an upper bound.
- Byte counts are a token proxy (bytes / 4). They say nothing about cache effects or model accuracy.
- The guard findings were shown on a small fixture crate. The mechanisms (missing libtest summary line; IL-3 reading the first word) do not depend on crate size.
- The live `git diff` ran on a working tree dirty with other sessions' edits, including an 800-line audit log. Absolute sizes are illustrative; what rtk removes is not.
- `rtk discover`'s 1.6M-token figure is rtk's own estimate and was not independently checked.

7. Recommendation

- 1. Don't wrap `run_command` with `rtk`.** The gain is at most 0.6% and would cost two safety guards and the lossless-buffer guarantee.
- 2. Port the one idea that pays:** summarise `cargo test` output even below the 10 KB inline threshold (the median is 1.3 KB and it is currently returned raw). Constraints: keep the raw output in `@cmd_*`; keep compiler warnings; keep libtest's `test result:` line so the empty-test-selection and partial-selection checks still fire. Expected effect: about -88% on ~320 calls/month, around 0.4% of total output. It is small but cheap, and it improves every codescout user's shell calls, not just ours.
- 3. Optional, low value:** `rtk`-style deduplication of repeated log lines in codescout's generic summariser. We did not measure it separately.
- 4. If Bash is ever re-enabled:** re-run `rtk discover`. The stock `rtk` hook would then apply to that path, which codescout's buffering does not cover.

8. How to reproduce

Install `rtk` into a scratch directory and turn telemetry off:

```
curl -sL -o rtk.tgz
https://github.com/rtk-ai/rtk/releases/download/v0.49.0/rtk-x86_64-unknown-linux-musl.tar.gz
tar xzf rtk.tgz && export RTK_TELEMETRY_DISABLED=1 && ./rtk --version
```

Coverage and replay. Read `input_json` and `output_json` for `tool_name='run_command'` from `.codescout/usage.db`. Note that `output_json` is an MCP content array, so parse `[0].text` as JSON. Then:

```
rtk rewrite "$cmd" # exit 0 = rewritable (coverage)
printf '%s' "$stdout" | rtk pipe -f git-diff # only when the call was inline (no output_id)
```

Guard probes, on any cargo crate:

```
run_command("cargo test no_such_test") # empty_test_selection warning attached
run_command("rtk cargo test no_such_test") # no warning
run_command("rtk cargo test 2>&1 | tail -2") # not blocked by IL-3; exit_code 0 on a red suite
```

The full findings are also stored in codescout memory `research/rtk-evaluation`.